Innovation Centre for Education

NCS
NihonCommunicationSolutions

RABBI ZIDNI 'ILMA

YENEPOYA
(Deemed to be University)

YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

Project – Low Level Design

On

Real-time Sentiment Analysis for Agriculture

SUBMITTED BY

Mohamed Suhaib U - 23BCDSBA13

BCA(Data Science and Big Data Analytycs)

UNDER THE GUIDANCE OF

Industry Mentor: sumit kumar shukla

Guided by: Raksha

TABLE OF CONTENTS

1. Introduction……..... 2

2. Scope of the Document…… 2

3. Intended Audience……... 3

4. System Overview…….. 3

5. System Design……...….. 4

5.1 Application Design…….…….. 4

5.2 Process Flow…….…… 4

5.3 Information Flow……....…… 6

6. API Catalogue……..…… 6

7. Data Models and Schemas……..…… 6

8. Application Programming Interfaces……..…….. 7

9. Security and Non-Functional Requirements……..…….. 7

10. References……..…… 7

1. Introduction

The Real-Time Sentiment Analysis for Agriculture system is a data-driven analytical platform designed to process, analyze, and visualize customer reviews related to agricultural products such as seeds, fertilizers, irrigation systems, pesticides, and farming tools.

In modern agriculture, customer feedback plays a critical role in evaluating product performance and identifying issues faced by farmers. However, this feedback is usually available in unstructured textual form, making it difficult to analyze manually.

This system addresses this challenge by using Natural Language Processing (NLP) techniques to automatically classify reviews into three categories: Positive, Negative, and Neutral. The system uses a hybrid approach combining VADER sentiment analysis and Text Blob polarity scoring to improve accuracy.

The platform performs multiple operations including data ingestion, text preprocessing, sentiment classification, aggregation, and visualization. The final output is presented in an interactive dashboard, enabling users to make informed decisions.

2. Scope of the Document

This Low-Level Design document provides a detailed explanation of the internal working of the sentiment analysis system.

It covers the following aspects:

- The document explains how input data is read from CSV files and validated. It describes the text preprocessing steps such as cleaning, tokenization, and stopword removal.
- It also explains how sentiment analysis is performed using NLP techniques and how results are classified into different categories.

- The document further describes how data is aggregated and visualized using charts and dashboards.
- The scope includes API communication between frontend and backend, as well as the overall data flow in the system.

The following aspects are not included in this system:

Database integration is not implemented. User authentication and login systems are not part of this project. The system does not include cloud deployment or real-time API integration.

3. Intended Audience

This document is intended for different types of users.

- Faculty members and project evaluators can use this document to assess the technical implementation and design quality of the project.
- Students can use this document to understand how sentiment analysis systems are designed and implemented using NLP techniques.
- Developers can refer to this document to extend or improve the system by adding new features.
- Industry professionals can analyze how such systems can be applied in real-world agricultural scenarios.

4. System Overview

The system operates as a structured pipeline that processes data step by step and converts raw textual input into meaningful insights.

The system is divided into three main layers.

- The Data Processing Layer is responsible for reading input data, cleaning the text, and performing sentiment analysis. This layer uses Python libraries such as Pandas, NLTK, TextBlob, and VADER.

- The API Layer is implemented using Flask. It handles file uploads, processes requests, and returns results in JSON format.
- The Presentation Layer is the frontend dashboard developed using HTML, CSS, and JavaScript. It displays results using charts and allows user interaction.

The system ensures that each layer works independently while maintaining smooth data flow between them.

5. System Design

5.1 Application Design

- The application follows a modular design where each module performs a specific function.
- The data ingestion module reads CSV files and converts them into structured data. The text preprocessing module cleans the data by removing unwanted characters and stop words.
- The sentiment analysis module calculates polarity and subjectivity scores and classifies each review.
- The aggregation module computes statistics such as the number of positive, negative, and neutral reviews.
- The API module manages communication between frontend and backend. The dashboard module displays results visually.
- This modular approach improves maintainability and scalability.

5.2 Process Flow

The system follows a sequential process flow.

- The process starts when the user uploads a CSV file containing reviews. The system validates the file and reads it using Pandas.

- The text data is then cleaned using NLP techniques such as tokenization and stopword removal.
- After cleaning, sentiment analysis is performed using VADER and TextBlob. Each review is assigned a polarity score and classified into a sentiment category.
- The results are aggregated to calculate overall statistics. These results are then sent to the frontend in JSON format.

Finally, the dashboard displays the results using charts and graphs.

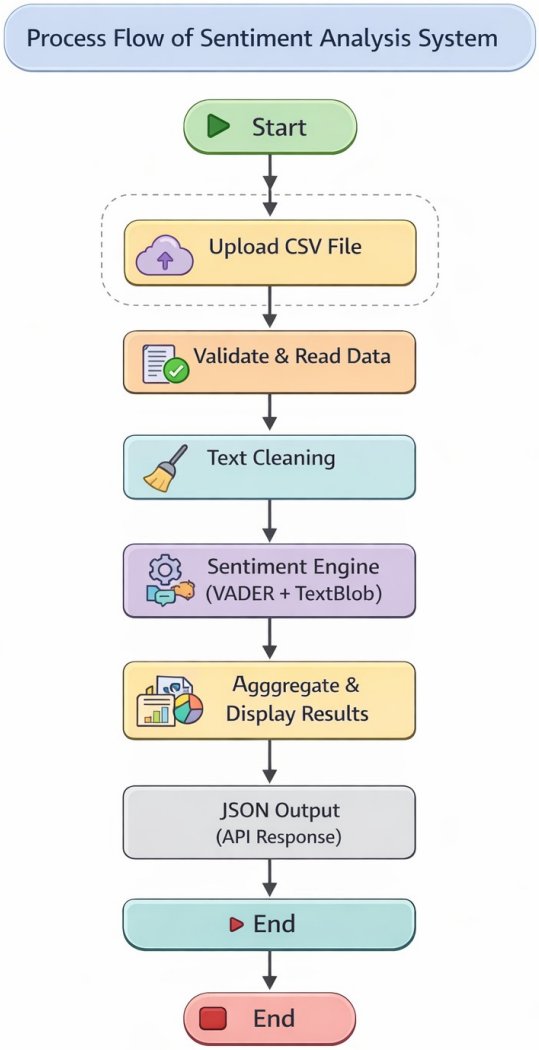

Figure 2: Process Flow of Sentiment Analysis System

5.3 Information Flow

The information flow describes how data moves through the system.

- The process begins with the CSV file input. The data is loaded into a DataFrame using Pandas.
- The text is cleaned and preprocessed. The cleaned text is passed to the sentiment analysis module.
- The results are classified and aggregated. The processed data is converted into JSON format.

This JSON data is sent to the frontend, where it is used to generate visualizations.

6. API Catalogue

- The system uses a simple API to handle communication between the frontend and backend.
- The main API endpoint accepts a CSV file and performs sentiment analysis.
- The API validates the file, processes the data, and returns the results in JSON format.
- This design ensures that the frontend and backend are loosely coupled and can be developed independently.

7. Data Models and Schemas

- The system uses structured data models for both input and output.
- The input data includes fields such as review ID, product name, review text, and category. These fields are read from the CSV file.
- After processing, additional fields are generated. These include cleaned text, polarity score, subjectivity score, and sentiment classification.

- These structured data models make it easier to perform analysis and visualization.

8. Application Programming Interfaces

- The system uses Flask to implement API functionality.
- The API accepts HTTP POST requests containing the CSV file. The backend processes the file and performs sentiment analysis.
- After processing, the API returns the results in JSON format.
- This allows the frontend dashboard to dynamically display data and update visualizations.

9. Security and Non-Functional Requirements

- The system is designed with performance, scalability, and usability in mind.
- The system can handle large datasets efficiently using optimized data processing libraries such as Pandas.
- The design is scalable, allowing future integration of advanced machine learning models.
- The system is user-friendly and provides a simple interface for uploading files and viewing results.
- Data security is ensured by performing all processing locally without sending data to external servers.

10. References

- Pandas Documentation (pandas.pydata.org)Used for reading CSV files, data manipulation, and DataFrame operations
- NLTK Documentation (nltk.org) Used for text preprocessing such as tokenization and stopword removal

- TextBlob Documentation (textblob.readthedocs.io)Used for calculating sentiment polarity and subjectivity
- VADER Sentiment Analysis (github.com/cjhutto/vaderSentiment) Used for rule-based sentiment classification with domain-specific words
- Flask Documentation (flask.palletsprojects.com) Used for building backend API and handling file uploads
- Chart.js Documentation (chartjs.org)Used for creating interactive charts in the dashboard
- Python Official Documentation(docs.python.org)Core programming language reference
- Kaggle (kaggle.com)Used for sample datasets and understanding real-world data
- Stack Overflow (stackoverflow.com) Used for debugging and resolving coding issues
- Matplotlib Documentation (matplotlib.org)Used for generating charts and visualizations